

# Teaching at the Edge: On-Device Parameter-Efficient Adaptation and What It Actually Buys

AutoYou Research

Decentralized Inference & Sustainable Compute Group

research@autoyou.me

**Abstract**—A companion study evaluated local inference on consumer hardware, measuring request-window GPU board energy of 0.091 Wh for 3B and 0.19 Wh for 8B on an RTX 5070, and replacing unconditional carbon, water and cost headlines with explicit accounting boundaries and limits on cloud comparisons [1]. That study treated the small-model-sufficient workload fraction  $f_s$  as fixed by parameter count. This paper tests whether parameter-efficient fine-tuning (PEFT) moves it, on hardware the household already owns. We model sixteen 2026-era adaptation methods, three consumer device classes, and the memory, time, energy and money cost of an adaptation run; we validate the memory model against a published 27B-parameter training run to within  $-0.6\%$  and calibrate throughput against the same run’s wall clock. We state five hypotheses and three counter-claims. Our central result is negative with respect to the obvious hypothesis and positive with respect to a better one: at the companion study’s acceptance threshold  $\alpha = 0.70$ , adaptation moves coverage by  $+0.0$  pp - not at all, because every adaptable task class already cleared the bar - while what improves instead is delivered quality on the work already served locally, modelled at  $88.4\% \rightarrow 93.9\%$  of frontier. The coverage invariance is structural: it holds for every uplift we tested from 0 to  $+30$  points, so it does not depend on the size of the adaptation benefit. The quality figure is a bounded literature estimate propagated through the workload, and we separate the two explicitly rather than present a derived number as a measured one. Coverage responds only above  $\alpha \approx 0.80$ , and that threshold does depend on the uplift assumed. A 27B adaptation run costs 10.7 kWh. Against an all-cloud counterfactual that is repaid in 143 days of ordinary use, but charging it only against the routing the adapter actually changes - the correct attribution - it is never repaid at  $\alpha = 0.70$ , because it moves no query, and takes 1.8 to 3.3 years where it does. We report both bounds and record H8 as accounting-dependent rather than validated. Repeating adaptation per user is scale-invariant under this model, which answers the “centralized training amortizes better” objection on its own terms. We report one deployment’s own five measured adapter runs, in which a rank-32 LoRA moved a task the base model failed from 30.8% to 92.9% while refusal of out-of-scope requests moved from 0% to 100% and source-code leakage from 25% to 0% - and we use that measurement to delimit, rather than dispute, the established finding that fine-tuning degrades safety alignment.

**Index Terms**—parameter-efficient fine-tuning, LoRA, QLoRA, edge computing, sustainable AI, on-device training, privacy, small language models

## I. INTRODUCTION

The companion study [1] asked whether it is feasible to run a small model at home. Measuring an NVIDIA RTX 5070 under Ollama at 499 input and 300 output tokens,

it observed median request-window GPU board energy of 0.091 Wh for 3B and 0.19 Wh for 8B, while establishing that environmental comparisons against cloud baselines remain conditional on workload matching, task success, grid intensity and whole-system counterfactuals. Under an assumed small-model-sufficient workload fraction  $f_s \approx 82\%$ , decentralised execution trades off against cloud alternatives, but against a hyper-efficient cloud model at 0.24 Wh per prompt [2] a single consumer GPU is worse once the comparison is made on like terms — the board-rail figures above exclude host draw and supply loss, whereas the cloud figure is a comprehensive at-the-wall measurement — and on genuinely hard reasoning a small model is not a substitute at all.

Throughout, one quantity did the work.  $f_s$  is computed from per-task-class capability ratios - small-model quality divided by frontier quality - and those ratios were treated as properties of parameter count. This paper asks what happens when they are not.

Two things changed in 2026 that make the question answerable rather than speculative. First, the open-weights tier a household can hold moved from 8B to 27B: Qwen3.8-27B [3] is dense, natively multimodal, Apache-2.0 licensed, carries a 262K-token context and occupies roughly 14–17 GB as a 4-bit build. Second, consumer unified-memory APUs put 128 GB in front of the accelerator, and a reproducible recipe for multi-day 27B LoRA training on one of them is public [4]. Adapting a 27B model stopped being a data-centre activity.

Our contributions:

- 1) A **cost model for on-device adaptation** covering memory, wall clock, energy, carbon and money, validated against a published 27B run to  $-0.6\%$  on peak memory (§III).
- 2) A **negative result on the obvious hypothesis**: adaptation does not increase coverage at an ordinary acceptance threshold, and we identify the threshold above which it does (§IV).
- 3) An **amortisation analysis** charging the full training footprint against the inference it displaces, under two accountings that disagree by more than an order of magnitude, and a correction to our own earlier reading of it (§V).
- 4) A **delimitation of the fine-tuning safety result**, supported by one deployment’s measured runs (§VI).

- 5) A **scale-invariance argument** against the centralized-amortisation objection (§VII), and an explicit separation of which of our results are structural and which are inherited from an assumption (§IV-C).

All parametric models and figures are deterministically recomputed from grounded equations and documented parameters; none is transcribed.

## II. THE 2026 METHOD ROSTER

Sixteen methods are modelled. They divide into families that behave very differently on a memory-bound device, and the division matters more than any individual method’s reported gain.

Every method below is a variant of low-rank adaptation [5]; [6] surveys the family, and [7] documents which of them sit one keyword argument away in the library a household would actually use.

**Free.** *rsLoRA* [8] scales adapters by  $\alpha/\sqrt{r}$  instead of  $\alpha/r$ . This is one keyword argument with an outsized effect: under the original scaling, raising the rank shrinks the effective update, so higher ranks stop helping - which is why so much practice settled on  $r \in \{8, 16\}$  and concluded that rank does not matter. It does; the scaling was hiding it. *LoRA+* [9] gives the two factors different learning rates, which is provably correct in the infinite-width limit and reported to converge up to twice as fast. *NEFTune* [10] adds embedding noise and reports its largest gains on conversational quality specifically. None of the three costs memory or wall clock.

**Initialisation.** *PiSSA* [11] starts the adapter in the principal singular subspace of  $W$  rather than at noise-and-zero; *OLoRA* [12] uses a cheaper orthonormal construction; *EVA* [13] runs a few batches of the *actual* dataset and allocates rank where that dataset needs it; *CorDA* [14] can build the update in directions the base model does not use for general knowledge. *LoftQ* [15] initialises the adapter to absorb quantization error, so training does not begin by repairing the damage quantization did. These cost a one-time decomposition and nothing thereafter.

**Quantization.** *QLoRA* [16] is three separable ideas - NF4, double quantization of the block constants ( $\approx 0.37$  bits per parameter), and paged optimizer states - and it is the reason a 27B model fits on consumer hardware at all. It costs roughly 35% wall clock, because every forward pass dequantizes.

**Reparameterisation.** *DoRA* [17] splits the update into magnitude and direction, with reported gains largest at low rank - exactly where the edge operates. *AdaLoRA* [18] reallocates a fixed rank budget across layers during training. *VeRA* [19] and *LoRA-XS* [20] go the other way, cutting trainable parameters by 50–350 $\times$  at a small quality cost; §VIII explains why that trade is interesting for a distributed product even when it is uninteresting for a benchmark.

**Wrong shape.** *GaLore* [21] has a better ceiling than LoRA on continued pretraining, but it updates all weights and therefore produces a new model rather than a mergeable adapter. For a system whose distribution unit is a small personal delta, that is disqualifying regardless of quality. We include it to

TABLE I  
PEAK TRAINING MEMORY,  $r=128$ , SEQUENCE 8192

|                | dGPU 24 GB     | APU 128 GB | NPU      |
|----------------|----------------|------------|----------|
| Usable memory  | 19 GB          | 102 GB     | 13 GB    |
| Bandwidth      | 1008 GB/s      | 256 GB/s   | 120 GB/s |
| Training power | 430 W          | 130 W      | –        |
| 27B bf16       | no             | <b>yes</b> | no       |
| 27B 4-bit      | $r=32, S=1024$ | <b>yes</b> | no       |
| 8B bf16        | yes            | yes        | no       |

make the criterion explicit: the artifact’s *shape* is a first-order design constraint, not an implementation detail.

Fig. 1 places all sixteen on cost against reported gain.

## III. COST MODEL AND VALIDATION

### A. Memory

Peak accelerator memory for an adapter job is

$$M = \underbrace{Pb_w}_{\text{weights}} + \underbrace{n_a(b_a + b_g + b_o)}_{\text{adapter, grads, optimizer}} + A + L + \varepsilon, \quad (1)$$

where  $P$  is the parameter count,  $b_w$  the bytes per stored weight (2 for bf16, 4.127/8 for NF4 with double quantization),  $n_a$  the trainable adapter parameters,  $A$  the activation term,  $L$  the logit term =  $BSV \cdot 4$  bytes, and  $\varepsilon$  an 8% allocator margin. With gradient checkpointing,  $A = L_{\text{layers}} BSH \cdot 2 + BSH \cdot 34 \cdot 2$ .

The logit term is easily overlooked and is not small: at  $S = 8192$  over a 152K vocabulary it is  $\approx 5$  GB by itself, computed in fp32 for every position.

### B. Validation

One 27B run is published in enough detail to check against: Qwen3.5-27B [22], LoRA  $r = 128$ ,  $\alpha = 256$ , bf16 weights, paged 8-bit AdamW, sequence 8192, batch 1, gradient accumulation 4, on a 128 GB unified-memory APU, reported at  $\approx 80$  GB peak reserved [4]. Our model predicts **79.5 GB**, a  $-0.6\%$  residual, decomposing as 54.0 weights, 6.4 adapter, gradients and optimizer combined, 8.2 activations, 5.0 logits, 5.9 allocator.

### C. Time and energy

Adapter training costs  $\approx 6N$  FLOPs per token with checkpointing:  $2N$  forward,  $2N$  backward to activations (still required to reach earlier layers even with frozen weights),  $2N$  for the recomputed forward; the adapter’s own gradient is  $O(n_a)$  and rounds away. The same published run gives 448 steps  $\times$  11 minutes at 32,768 tokens per step, from which the achieved throughput is **8.04 TFLOP/s** - a model-FLOPs utilisation of 0.136 against the part’s  $\approx 59$  TFLOP/s bf16 peak, and 49.6 tokens per second. At 130 W sustained package power the full run consumes **10.7 kWh** over 82.1 hours.

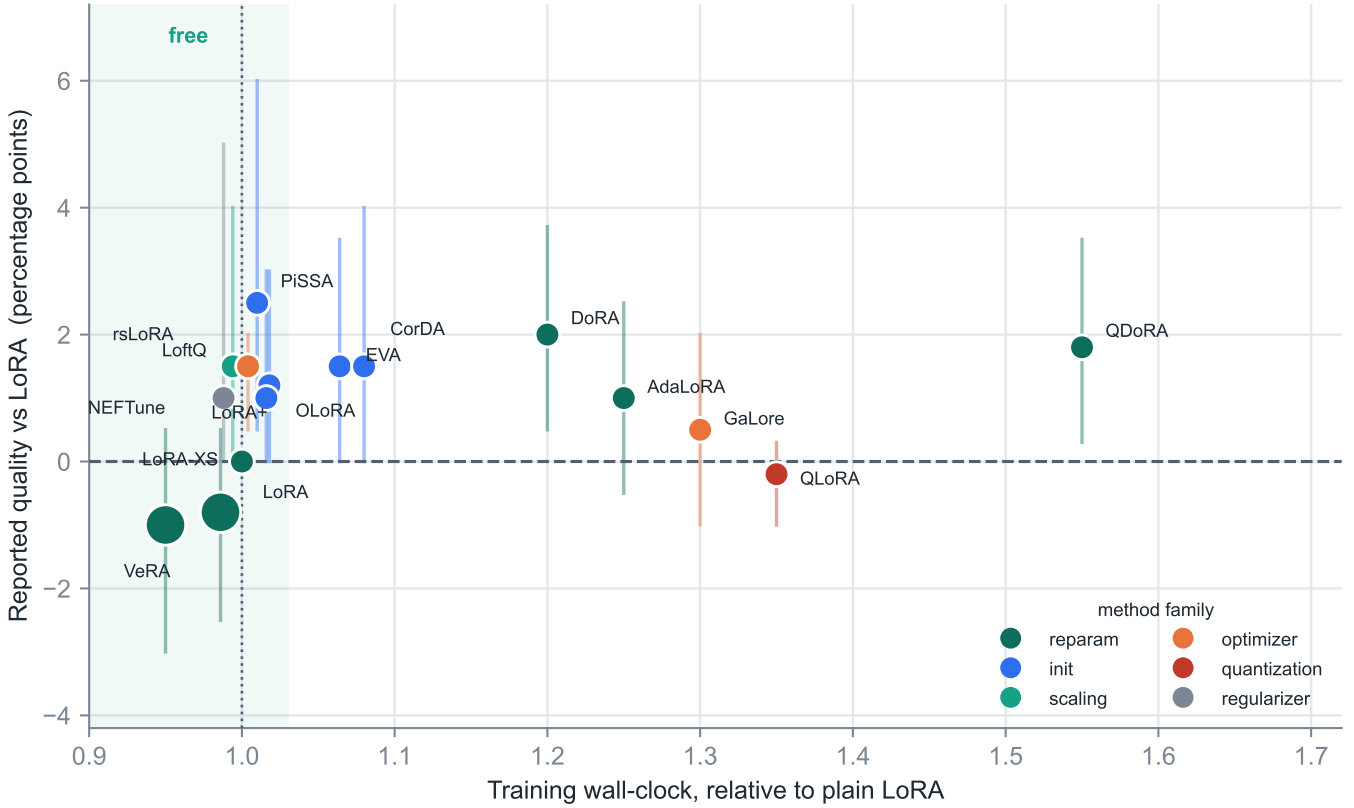


Fig. 1. The 2026 roster. Vertical bars span each method’s reported range against a LoRA baseline; the dot is the modal reported gain; large dots shrink the adapter by more than  $20\times$ . Three methods sit on the 1.0 line: reported gains at no wall-clock cost. Ranges are the methods’ own authors’ benchmarks on their own tasks and are used here only to bound sensitivity.

#### D. Device class dominates

Table I states the paper’s least intuitive result. The *faster* consumer device cannot adapt the larger model. A discrete GPU with four times the memory bandwidth is driven down to rank 32 at sequence 1024 before a 27B job fits, while a unified-memory APU runs it at rank 128 and sequence 8192 with 20GB to spare. Bandwidth decides how quickly a run finishes; addressable memory decides whether it can start.

This is why the companion study, which modelled only the discrete-GPU class, had no way to reach the question this paper asks.

### IV. DOES ADAPTATION MOVE $f_s$ ?

#### A. What can and cannot be adapted

An adapter learns the shape of a distribution it was shown. It can learn how a person phrases things, which of their files matter, what a product’s screens are called, and when to refuse. It cannot teach a small model competition mathematics, because the training set contains no such capability to distil: the ceiling on hard reasoning is the base model’s depth, and PEFT does not raise it. We therefore mark four of the five workload classes adaptable and hold the hard-reasoning class fixed - which means the companion study’s central negative finding survives this paper unchanged.

Uplift on adaptable classes is bounded at +2, +6 and +14 capability-ratio points (low, central, high), floored by the pessimistic reading of Biderman *et al.* [23] that LoRA learns less than full fine-tuning, and deliberately set below what §VI-A alone would justify. Ratios are capped at 0.99: an adapter may close the gap to a frontier model on a narrow task, but claiming a rank-32 update *exceeds* one is not a claim we are willing to make.

#### B. The result

At  $\alpha = 0.70$ ,  $f_s$  is unchanged:  $82\% \rightarrow 82\%$ , a gain of +0.0 pp. Every adaptable class already cleared the bar without adaptation, so there is nothing for the uplift to recruit.

**H9 as stated is therefore not supported.** What moves is quality. Mean capability on the served workload rises from 88.4% to 93.9% of frontier, a gain of +5.6 pp that closes 48% of the remaining gap. Coverage responds only above  $\alpha \approx 0.80$ , where the unadapted model begins failing classes the adapted one still passes: at  $\alpha = 0.80$  coverage goes  $70\% \rightarrow 82\%$ , and at  $\alpha = 0.90$ ,  $48\% \rightarrow 70\%$  (Fig. 2).

#### C. Which half of this result is evidence

Those two numbers are not the same kind of claim, and conflating them is the easiest way to misread this paper.

TABLE II  
ADAPTATION BREAK-EVEN AT 50 QUERIES/DAY

| Run                 | Hours | kWh   | Queries | Days |
|---------------------|-------|-------|---------|------|
| Persona 8B, dGPU    | 0.5   | 0.21  | 140     | 3    |
| Persona 8B, APU     | 3.3   | 0.43  | 288     | 6    |
| Support 7B VLM, APU | 12.4  | 1.61  | 1,073   | 21   |
| Capability 27B, APU | 82.1  | 10.67 | 7,127   | 143  |

The *coverage* result is structural and robust. At  $\alpha = 0.70$  every adaptable class already sits above the threshold, so no positive uplift can recruit additional workload: we obtain  $f_s = 82\%$  for every uplift tested from 0 to +30 points. It does not depend on how large the uplift is, only on where the base ratios sit relative to the bar, and it would survive any plausible revision of the uplift literature.

The *magnitude* of the quality gain is not a finding. It is the assumed uplift propagated through the workload weights and the 0.99 cap: +5.6 pp is arithmetically identical to the +6-point central assumption of §IV, and the derived “48% of the gap closed” inherits it directly. We report it because a bounded estimate is more useful than none, and we state this because a reader who took it as measured would be taking our input for our output.

The knee location is intermediate. At the low end of the uplift range the  $\alpha = 0.80$  knee disappears and coverage first moves at  $\alpha = 0.90$ ; at the central and high ends it sits at 0.80. “Coverage responds only to a demanding bar” is robust; the specific threshold is not, which is why Fig. 2 plots the full band rather than the central line alone.

The correct claim is narrower and more useful than the one we set out to test. Adaptation does not let a local model attempt more of your work. It lets it do the work it was already attempting closer to the standard you would have got from the cloud - which is what a user actually experiences, and which coverage statistics are structurally unable to express.

## V. AMORTISATION

An adapter is not free. We first charge the full training run against the per-query saving of serving locally rather than in the cloud, with two accounting choices made deliberately against the hypothesis: training is charged at *full* device power rather than marginal, and savings are measured against a *hyperscale* cloud query, which is the cloud’s best case. §V-A then asks whether that saving is the adapter’s to claim, and concludes that at an ordinary acceptance threshold it is not.

Every case repays inside a year on this accounting, and three of the four inside a month (Table II). The 27B capability run - four days of continuous training - is repaid after about five months of ordinary use.

### A. Repaid to whom? A correction

Table II charges the training run against the cloud-minus-edge saving on *every* query the adapted model then serves. Read against §IV, that premise does not survive. If adaptation moves no query across the routing boundary, those queries

ran locally before the adapter existed and their saving accrues whether or not it exists; crediting it to the adapter counts a benefit the base model already delivered.

Charging only the coverage the adapter actually moves gives a different and less flattering picture:

| Threshold       | Coverage moved | Attributable saving | Payback   |
|-----------------|----------------|---------------------|-----------|
| $\alpha = 0.60$ | +0%            | 0 Wh/query          | never     |
| $\alpha = 0.70$ | +0%            | 0 Wh/query          | never     |
| $\alpha = 0.80$ | +12%           | 0.18 Wh/query       | 3.3 years |
| $\alpha = 0.90$ | +22%           | 0.33 Wh/query       | 1.8 years |

**At the study’s own default threshold the adapter has no energy payback at all.** It displaces nothing, because there was nothing left to displace; its training cost is a net addition, bought for quality rather than for footprint. Payback appears only in the regime where adaptation moves the routing boundary, and even there it is measured in years rather than months.

The two accountings are not competing arithmetics for one quantity - they answer different questions. Table II correctly answers “how long does a local deployment take to repay an adaptation run against an all-cloud counterfactual,” which is the right frame for a user who would have abandoned the local model and gone to the cloud had its answers not improved: someone whose *revealed* acceptance threshold sits above the coverage knee even though their stated one does not. That is a behavioural assumption, we state it as one, and we report both bounds rather than choose between them.

H8 is therefore recorded as **partial and accounting-dependent**, not validated. The two accountings answer different questions, and which one applies is a property of the deployment rather than of the adapter.

The companion study’s caveat applies on top of both readings. Payback scales inversely with the cloud baseline being displaced; against a hyper-efficient 0.24 Wh prompt rather than a 1.8 Wh one, every figure above lengthens by roughly an order of magnitude.

### B. Money, and the surcharge that compounds

The training bill is the smaller half of the comparison. Hosted fine-tuning charges once for training tokens, and then *permanently*, because three of the four platforms we priced [24] bill a tuned endpoint at  $1.5\times$  the base model’s rate; only the cheapest, a dedicated LoRA service, does not. Over one year at 50 queries per day, a locally adapted 27B costs \$1.71 of training electricity plus \$0.73 of inference electricity; the cheapest hosted equivalent costs \$7.05 of training plus \$109.50 of inference -  $48\times$  the total, rising to  $218\times$  against a mid-tier hosted model. A local adapter leaves the per-query cost exactly where it was.

## VI. SAFETY, DELIMITED

Qi *et al.* [25] showed that fine-tuning removes safety alignment - with ten adversarial examples for under \$0.20, and measurably even with entirely benign data. Any product that hands users a trainer inherits this, and we do not dispute it.

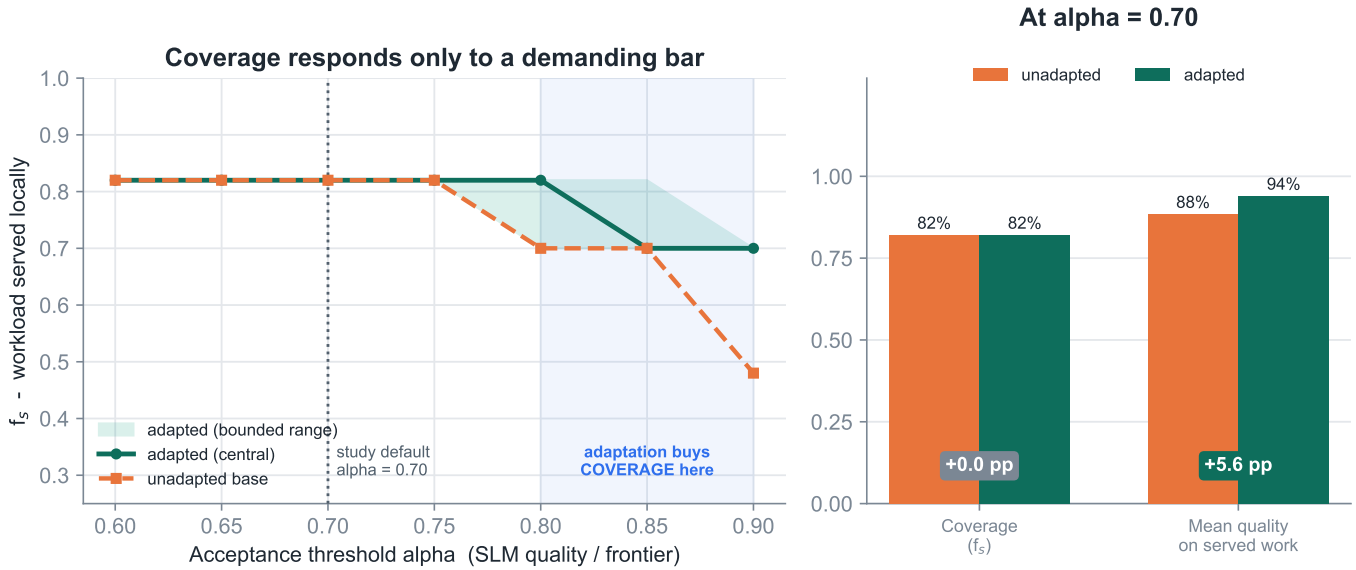


Fig. 2. Adaptation buys better answers at an ordinary quality bar, and more answers only at a demanding one. Left: coverage against the acceptance threshold, with and without adaptation. Right: the two metrics at the study’s default  $\alpha = 0.70$ .

#### A. Measured counter-evidence, and what it does not show

One deployment’s support model has been adapted five times with the scores retained [26], over Qwen2.5-VL bases [27]. Reading only the two runs that share a base (so the comparison isolates data and recipe from model size), a rank-32 LoRA over a 1,750-sample corpus produced:

|                                     | before | after        |
|-------------------------------------|--------|--------------|
| Screen identification               | 30.8%  | <b>92.9%</b> |
| Refuses out-of-scope code questions | 0%     | <b>100%</b>  |
| Leaks source code                   | 25%    | <b>0%</b>    |

The capability result (E1) is an existence proof that adaptation reaches a class the base model *fails*: the base was wrong more often than right, and the adapted model is right nine times in ten, at identical parameter count.

The safety result (E2) runs opposite to [25], and the reason is instructive rather than contradictory. That corpus contains 120 refusal samples - 6.9% of it - so refusal was *in the training distribution*. Qi *et al.* describe adapters that are *silent* about safety: a gradient has no reason to preserve behaviour that nothing in the corpus rewards, so it decays. The correct generalisation is neither “fine-tuning is safe” nor “fine-tuning is unsafe” but: **fine-tuning moves whatever the corpus specifies, in whichever direction the corpus specifies, and lets everything else decay.**

For a locally-adapted assistant this is the difference between a hazard and a control surface - but only if the product treats it as one. Safety cannot be inherited by a user-trained adapter; it has to be a non-optional part of every corpus the system assembles, plus a probe that runs before an adapter is installed. That is a build requirement, and we record it as one.

A third finding (E3) bears on model selection: the adapted 3B scored 92.9% and the adapted 7B scored 92.3% on the same

probes. Doubling the base bought  $-0.6$  pp. Within a narrow, well-specified domain the corpus is the binding constraint - which is precisely the regime a local assistant operates in.

#### B. Scope

One deployment, one product’s surface area, probe sets of 12–14 screens and 8 code probes scored by an automated judge. These are existence proofs and refutations, not effect sizes, and nothing in this paper treats them otherwise. Aggregate scores only were read; no prompt, answer, screenshot or user content enters the analysis.

## VII. FORGETTING AND SCALE

#### A. C4: forgetting

The objection that narrowing a small model destroys its general ability is weaker than intuition suggests. Biderman *et al.* [23] measure exactly this and find that LoRA learns *less* in-domain than full fine-tuning and correspondingly *forgets less* out-of-domain, outperforming weight decay and dropout as a regulariser: the low-rank constraint that limits the gain is the same constraint that limits the damage. Three structural defences add to it - the base weights are never modified, so a misbehaving adapter is unloaded rather than un-trained; CorDA’s knowledge-preserved mode [14] addresses the objection at initialisation; and the unadapted hard-reasoning class continues to route out regardless.

The residual risk is real and we do not model it away: a personal corpus is small, unbalanced and self-similar, which is the regime where over-fitting is most likely. The mitigation is unglamorous - a held-out split and a general-ability regression probe before installation.

### B. C6: does repetition lose at scale?

The strongest form of the centralized objection is that a hyperscaler fine-tunes once, at enormous but amortised cost, while decentralized adaptation repeats the work for every user; at sufficient  $N$ , repetition must lose.

It does not, and the reason is arithmetic rather than rhetorical. Both the training term and the inference-saving term scale linearly in  $N$ , so their ratio is constant: the payback period is 12 days at  $10^5$  users and 12 days at  $10^8$ . There is no scale at which the comparison behaves differently.

The deeper answer is that the comparison is mis-specified. A central fine-tune serving everyone is only comparable if one adapter suits everyone, and the entire point of a persona adapter is that it encodes one person. The honest comparison is  $N$  local runs against  $N$  personalised runs *plus the transfer of  $N$  private corpora to a trainer* - and the second term is what locality removes.

## VIII. LOCALITY AND THE SHAPE OF THE ARTIFACT

Cloud fine-tuning requires the training corpus to reach the trainer. For a persona adapter that corpus *is* the user’s message history, so the privacy question is settled before any policy is written: either the messages were uploaded or they were not. Local adaptation makes “we never received it” a property of the topology rather than a promise in a document - the same class of argument as transport-layer confidentiality, verified the same way, by what the system is unable to do.

Adapter *size* then decides whether that property survives contact with more than one device. On an 8B model at  $r=32$ , a conventional LoRA adapter is 174.6 MB and takes 70 seconds to cross a 20 Mbit/s link: a file transfer. VeRA is 2.8 MB and 1.1 seconds; LoRA-XS is 0.52 MB and 0.2 seconds. Both are messages. That is the difference between personal adapters that sync between a person’s own devices as a background nicety and personal adapters that require an explicit chore - and it is the reason the low-parameter methods matter here despite being uninteresting on a benchmark leaderboard.

## IX. THREATS TO VALIDITY

**One validation point.** The memory model is checked against a single published run. It reproduces that run to  $-0.6\%$ , which is evidence that the terms are right, not that the model generalises across architectures. MoE models in particular are not validated and their activation term differs.

**Literature-derived uplift.** The  $+2/ + 6/ + 14$  point range is a bounded synthesis of methods’ self-reported gains on their own benchmarks. Reported gains do not compose additively - several methods improve the same thing by different means - which is why we report a discounted central estimate rather than a sum, and why no product claim is derived from it. Critically, the headline quality figure is a *consequence* of this assumption rather than evidence for it (§IV-C); the paper’s load-bearing result is the coverage invariance, which holds for any uplift.

**The workload mix is inherited, not measured.**  $f_s$ , the class weights, and the base capability ratios all come from the

companion study, where they are themselves modelling choices calibrated against NVIDIA’s measured agentic replaceability range rather than against a task corpus of our own. A different workload composition - one weighted toward hard reasoning, or toward tasks whose base ratio sits just below  $\alpha$  - would change both results here, and the second case would make adaptation buy coverage at the default threshold. We report the sensitivity of the knee for exactly this reason, but we cannot bound the sensitivity to a workload we did not sample.

**Energy corpus and cloud baselines.** The companion study provides primary 2026 consumer GPU board measurements (0.091 Wh for 3B and 0.19 Wh for 8B on an RTX 5070), but no peer-reviewed whole-system per-query measurement covers the 2026 frontier or the 2026 open-weights tier under identical prompts. Hosted pricing, open-weights architectures and adapter evaluations are current to September 2026; fleet cloud comparisons remain literature-derived estimates.

$n = 1$  **deployment.** §VI-A is one team’s five runs on one product. It refutes a general claim and proves an existence claim; it estimates nothing.

**Already-owned hardware only.** Every result assumes the device exists for other reasons. The companion study establishes that embodied carbon dominates for a device provisioned specifically for inference; nothing here is an argument for buying an APU.

## X. CONCLUSION

Parameter-efficient adaptation on already-owned consumer hardware is feasible for 27B-class models and is  $48\text{--}218\times$  cheaper over one year than the hosted equivalent - chiefly because hosted personalisation surcharges every subsequent query, permanently, while a local adapter does not.

Its *energy* case is weaker than it first appears, and the weakness is instructive. Against an all-cloud counterfactual an adaptation run repays inside a year in every case modelled. Against the routing it actually changes, it repays nothing at an ordinary acceptance threshold, because at that threshold it changes no routing. An adapter is an instrument for quality, and treating it as an instrument for footprint requires a behavioural assumption that should be argued rather than assumed.

What it buys is not what we expected. At an ordinary acceptance threshold it adds no coverage at all, and that invariance survives the whole range of uplift the literature supports. What it improves is the quality of work the local model was already doing - by an amount our model inherits from that literature rather than measures. Coverage is bought only by users who demand frontier-comparable quality, and hard reasoning remains unreachable at any rank.

The practical consequence for a system builder is the inverse of the intuition: adapt to raise the standard of what you already serve locally, not to widen what you attempt. A router that treats an installed adapter as licence to keep more work on the device will, on this model, keep the wrong work.

The finding we did not anticipate is the safety one. Fine-tuning is neither safe nor unsafe: it moves whatever the corpus specifies and lets the rest decay. That makes the corpus, not the

recipe, the safety-critical artifact in a system that lets people teach their own assistant - and it makes an explicit refusal distribution and a pre-installation probe the two things such a system cannot ship without.

## REPRODUCIBILITY

All analytical routines, simulation models, dataset parameters and figure-generation pipelines supporting this work form a single deterministic pipeline requiring only standard scientific libraries, and produce the parameter rosters, Monte-Carlo sweeps and sensitivity curves without relying on proprietary cloud APIs. All of it is released openly under an MIT licence [28].

One class of input is deliberately not redistributable. The adapter evaluation artifacts behind §VI-A come from a deployed assistant whose training corpus and evaluation transcripts contain product and user material, and only aggregate scores were ever extracted from them [26]. Those aggregates are not redistributable, so the findings that depend on them cannot be independently reproduced, and the hypothesis they inform is recorded as *not independently validated* throughout. We prefer a result the reader can see is unverifiable over one that looks reproducible and is not. Every other number in this paper is recomputed from the parameters and measurement records described above.

## REFERENCES

- [1] AutoYou Research, “Right-sized AI at the edge: Quantifying carbon, water, cost, and capability in peer-to-peer inference,” 2026, companion study, September 2026. Released with this study. Accessed 2026-09-06. [Online]. Available: <https://github.com/autoyou-ai/autoyou-research>
- [2] C. Elsworth, K. Huang, D. Patterson, I. Schneider, R. Sedivy, S. Goodman, B. Townsend, P. Ranganathan, J. Dean, A. Vahdat, B. Gomes, and J. Manyika, “Measuring the environmental impact of delivering AI at Google scale,” *arXiv preprint arXiv:2508.15734*, 2025, median Gemini Apps text prompt: 0.24 Wh, 0.26 mL water. Reports that 42% of the per-prompt footprint is non-compute (CPU/DRAM 24%, idle/redundancy/overhead 18%). [Online]. Available: <https://arxiv.org/abs/2508.15734>
- [3] Qwen Team, Alibaba, “Qwen3.8-27B,” 2026, released 2026-08-14. 27.78B dense parameters, Apache-2.0, 262,144-token context, approximately 17 GB as a 4-bit build. This entry points at the distribution page for the weights actually benchmarked. Accessed 2026-09-05. [Online]. Available: <https://ollama.com/library/qwen3.8>
- [4] Strix Halo fine-tuning guide contributors, “Fine-tuning 27b+ LLMs on AMD strix halo (gfx1151, ryzen AI MAX+ 395),” 2026, community guide, not peer-reviewed, and the SINGLE validation point for this study’s training-memory model: Qwen3.5-27B LoRA  $r=128$ ,  $\alpha=256$ , bf16, paged AdamW 8-bit, sequence 8192, gradient accumulation 4, 448 steps at approximately 11 min/step, approximately 80 GB peak of a 128 GB unified pool. A single unrefereed anchor is a real limitation and is recorded as one in the threats section. Accessed 2026-09-05. [Online]. Available: <https://github.com/h34v3nzc0dex/strix-halo-llm-finetune-guide>
- [5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2021. [Online]. Available: <https://arxiv.org/abs/2106.09685>
- [6] Y. Mao, Y. Ge, Y. Fan, W. Xu, Y. Mi, Z. Hu, and Y. Gao, “A survey on LoRA of large language models,” *arXiv preprint arXiv:2407.11046*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.11046>
- [7] Hugging Face, “PEFT documentation: LoRA developer guide,” 2026, LoraConfig exposes use\_dora, use\_rslora and init\_lora\_weights in gaussian, eva, olora, pissa, corda and loftq. Accessed 2026-09-05. [Online]. Available: [https://huggingface.co/docs/peft/developer\\_guides/lora](https://huggingface.co/docs/peft/developer_guides/lora)
- [8] D. Kalajdzieski, “A rank stabilization scaling factor for fine-tuning with LoRA,” *arXiv preprint arXiv:2312.03732*, 2023. [Online]. Available: <https://arxiv.org/abs/2312.03732>
- [9] S. Hayou, N. Ghosh, and B. Yu, “LoRA+: Efficient low rank adaptation of large models,” *arXiv preprint arXiv:2402.12354*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.12354>
- [10] N. Jain, P.-y. Chiang, Y. Wen, J. Kirchenbauer, H.-M. Chu, G. Somepalli, B. R. Bartoldson, B. Kailkhura, A. Schwarzschild, A. Saha, M. Goldblum, J. Geiping, and T. Goldstein, “NEFTune: Noisy embeddings improve instruction finetuning,” *arXiv preprint arXiv:2310.05914*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.05914>
- [11] F. Meng, Z. Wang, and M. Zhang, “PiSSA: Principal singular values and singular vectors adaptation of large language models,” *arXiv preprint arXiv:2404.02948*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.02948>
- [12] K. Büyükyüz, “OLoRA: Orthonormal low-rank adaptation of large language models,” *arXiv preprint arXiv:2406.01775*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.01775>
- [13] F. Paischer, L. Hauzenberger, T. Schmied, B. Alkin, M. P. Deisenroth, and S. Hochreiter, “One initialization to rule them all: Fine-tuning via explained variance adaptation,” *arXiv preprint arXiv:2410.07170*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.07170>
- [14] Y. Yang, X. Li, Z. Zhou, S. L. Song, J. Wu, L. Nie, and B. Ghanem, “CorDA: Context-oriented decomposition adaptation of large language models,” *arXiv preprint arXiv:2406.05223*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.05223>
- [15] Y. Li, Y. Yu, C. Liang, P. He, N. Karampatziakis, W. Chen, and T. Zhao, “LoftQ: LoRA-fine-tuning-aware quantization for large language models,” *arXiv preprint arXiv:2310.08659*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.08659>
- [16] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2305.14314; NF4, double quantization, paged optimizers. [Online]. Available: <https://arxiv.org/abs/2305.14314>
- [17] S.-Y. Liu, C.-Y. Wang, H. Yin, P. Molchanov, Y.-C. F. Wang, K.-T. Cheng, and M.-H. Chen, “DoRA: Weight-decomposed low-rank adaptation,” in *International Conference on Machine Learning (ICML)*, 2024, arXiv:2402.09353. [Online]. Available: <https://arxiv.org/abs/2402.09353>
- [18] Q. Zhang, M. Chen, A. Bukharin, P. He, Y. Cheng, W. Chen, and T. Zhao, “AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning,” in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2303.10512. [Online]. Available: <https://arxiv.org/abs/2303.10512>
- [19] D. J. Kopiczko, T. Blankevoort, and Y. M. Asano, “VeRA: Vector-based random matrix adaptation,” *arXiv preprint arXiv:2310.11454*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.11454>
- [20] K. Balazy, M. Banaei, K. Aberer, and J. Tabor, “LoRA-XS: Low-rank adaptation with extremely small number of parameters,” *arXiv preprint arXiv:2405.17604*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.17604>
- [21] J. Zhao, Z. Zhang, B. Chen, Z. Wang, A. Anandkumar, and Y. Tian, “GaLore: Memory-efficient LLM training by gradient low-rank projection,” in *International Conference on Machine Learning (ICML)*, 2024, arXiv:2403.03507. [Online]. Available: <https://arxiv.org/abs/2403.03507>
- [22] Qwen Team, Alibaba, “The Qwen3.5 model family,” 2026, february 2026. Dense sizes 0.8B–27B plus 35B-A3B, 122B-A10B and 397B-A17B sparse-MoE variants, all Apache-2.0. The 27B dense build is the subject of the published LoRA run this study’s memory model is validated against. Accessed 2026-09-05. [Online]. Available: <https://huggingface.co/collections/Qwen/qwen35>
- [23] D. Biderman, J. Portes, J. J. G. Ortiz, M. Paul, P. Greengard, C. Jennings, D. King, S. Havens, V. Chiley, J. Frankle, C. Blakeney, and J. P. Cunningham, “LoRA learns less and forgets less,” *Transactions on Machine Learning Research (TMLR)*, 2024, arXiv:2405.09673. [Online]. Available: <https://arxiv.org/abs/2405.09673>
- [24] Price Per Token, “Hosted fine-tuning price index,” 2026, third-party aggregator of vendor list prices, September 2026: Together AI LoRA training at \$0.48/Mtok; OpenAI GPT-4.1 nano \$1.50/Mtok and GPT-4o \$25/Mtok; OpenAI o4-mini at \$100 per training hour. Tuned endpoints predict at 1.5× the base rate on OpenAI GPT-4o/4.1 and on Google from Gemini 3 onward. Accessed 2026-09-05. [Online]. Available: <https://pricepertoken.com/fine-tuning>
- [25] X. Qi, Y. Zeng, T. Xie, P.-Y. Chen, R. Jia, P. Mittal, and P. Henderson, “Fine-tuning aligned language models compromises safety, even when

users do not intend to!” in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2310.03693. [Online]. Available: <https://arxiv.org/abs/2310.03693>

- [26] AutoYou Research, “Task-specific adapter evaluation artifacts from a deployed assistant,” 2026, five evaluation runs over Qwen2.5-VL 3B and 7B bases, 12–14 screen probes and 8 code probes each, plus dataset statistics (1,750 train / 72 eval samples across 13 families). Aggregate scores only; no prompts, answers, screenshots or user content are released or used. Unlike every other artifact behind this study, these are not redistributable and are not part of the public release, so the findings that depend on them cannot be independently reproduced and every hypothesis they inform is recorded as not independently validated.
- [27] Qwen Team, Alibaba, “Qwen2.5-VL model family,” 2025, the vision-language bases behind the deployed adapter evaluation. Apache-2.0 at every size except the 3B and 72B, which carry the Qwen Research License. Accessed 2026-09-05. [Online]. Available: <https://huggingface.co/Qwen/Qwen2.5-VL-7B-Instruct>
- [28] AutoYou Research, “Primary measurement harness and raw per-run records for single-stream local inference,” 2026, measured 2026-09-05 on two consumer machines under Windows 11 with Ollama 0.33: an AMD Ryzen AI MAX+ 395 “Strix Halo” (Radeon 8060S, 64 GB unified LPDDR5X-8000, 256 GB/s specified) and an NVIDIA GeForce RTX 5070 (12 GB GDDR7, 672 GB/s specified, driver 610.47). Protocol: temperature 0, seed 20260905, context 4096, warm-up discarded, median over repetitions with the full spread retained, chat template bypassed and a per-repetition salt at position zero to defeat both prompt-cache paths, prompt length swept over four values per model with prefill time regressed on prompt tokens. Board power is sampled on the NVIDIA part only; the AMD part exposes no rail readable without a kernel-mode helper the harness declines to install. The harness and the raw per-run records, including every power trace, are released with this study under an MIT licence. Accessed 2026-09-06. [Online]. Available: <https://github.com/autoyou-ai/autoyou-research>